іКИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ТАРАСА ШЕВЧЕНКА
Факультет комп'ютерних наук та кібернетики
Кафедра теоретичної кібернетики

Курсова робота

за спеціальністю 122 Комп'ютерні науки

на тему:

РОЗРОБЛЕННЯ ВЕБ-ЗАСТОСУНКУ ДЛЯ АНАЛІЗУ ТА ВІЗУАЛІЗАЦІЇ УСПІШНОСТІ СТУДЕНТІВ

Виконала студентка 3-го курсу групи ТК-32

ОПП «Інформатика»

Савченко Олександра Віталіївна _____

(підпис)

Науковий керівник:
асистент кафедри теоретичної кібернетики
Ляшко Андрій Володимирович _____

(підпис)

Засвідчую, що в цій роботі немає запозичень з праць інших авторів без відповідних посилань

Студент

Київ – 2026

РЕФЕРАТ

Обсяг роботи 37 сторінок, 15 ілюстрацій, 8 таблиць, 10 джерел посилань, 2 додатки.

ВЕБ-ЗАСТОСУНОК, АНАЛІЗ УСПІШНОСТІ СТУДЕНТІВ, ВІЗУАЛІЗАЦІЯ ДАНИХ, PYTHON, STREAMLIT, РЕЛЯЦІЙНА БАЗА ДАНИХ, SQLITE, POSTGRESQL, АВТОРИЗАЦІЯ, ХЕШУВАННЯ ПАРОЛІВ, ЛІНІЙНА РЕГРЕСІЯ, ПОЛІНОМІАЛЬНА РЕГРЕСІЯ, ПРОГНОЗУВАННЯ УСПІШНОСТІ.

Об'єктом роботи є процес аналізу, обробки та візуалізації даних про академічну успішність студентів вищих навчальних закладів засобами веб-застосунку.

Предметом роботи є веб-застосунок, що забезпечує завантаження табличних даних у форматах CSV та XLSX, їх зберігання у нормалізованій реляційній базі даних, статистичний аналіз, інтерактивну візуалізацію та прогнозування успішності за допомогою методів машинного навчання.

Метою роботи є розроблення повнофункціонального веб-застосунку для моніторингу успішності студентів з підтримкою офлайн-режиму, системою авторизації з розмежуванням прав доступу та модулем прогнозування на основі регресійного аналізу.

Методи розроблення: проєктувально-розробницький підхід; методи нормалізації реляційних баз даних (3НФ); статистичний аналіз часових рядів; лінійна та поліноміальна регресія (метод найменших квадратів). Інструменти: Python 3.12, Streamlit 1.32, Pandas 2.0, Plotly 5.18, SQLite 3, PostgreSQL 15, SQLAlchemy 2.0, scikit-learn 1.4, ReportLab 4.0.

Результати роботи: розроблено повнофункціональний веб-застосунок з нормалізованою реляційною БД (5 таблиць із зовнішніми ключами), системою авторизації на основі SHA-256 хешування та моделлю RBAC (дві ролі), вісьмома типами інтерактивних графіків Plotly, чотирма режимами прогнозування успішності (лінійна та поліноміальна регресія), підтримкою офлайн-режиму на SQLite та синхронізацією з хмарним PostgreSQL. Проведено тестування у 15 тест-кейсах. Точність лінійної регресії: $R^2 = 0.181$.

Застосунок призначений для використання у вищих навчальних закладах як інструмент автоматизованого моніторингу та прогнозування академічної успішності студентів. Перспективи розвитку: REST API, мобільний клієнт, розширення методів прогнозування (ARIMA, градієнтний бустинг).

ЗМІСТ

ПЕРЕЛІК СКОРОЧЕНЬ ТА УМОВНИХ ПОЗНАК 5

ВСТУП 6

1 АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА ОГЛЯД ІСНУЮЧИХ РІШЕНЬ 9

1.1 Сучасний стан систем моніторингу успішності студентів 9

2 ПРОЄКТУВАННЯ СИСТЕМИ 12

2.1 Функціональні та нефункціональні вимоги 12

2.1.1 Функціональні вимоги 12

2.1.2 Нефункціональні вимоги 13

2.2 Архітектура застосунку та модульна декомпозиція 13

2.3 Проєктування бази даних 15

3 РЕАЛІЗАЦІЯ ЗАСТОСУНКУ 17

3.1 Система авторизації та розмежування прав доступу 17

3.2 Модуль завантаження та обробки даних 20

3.3 Модуль аналітики та інтерактивної візуалізації 21

3.4 Модуль прогнозування успішності 24

3.5 Модуль синхронізації з PostgreSQL 28

3.6 Модуль експорту звітів 29

4 ТЕСТУВАННЯ ТА АНАЛІЗ РЕЗУЛЬТАТІВ 30

ВИСНОВКИ 33

ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАННЯ 35

ДОДАТОК А 36

ДОДАТОК Б 38

ПЕРЕЛІК СКОРОЧЕНЬ ТА УМОВНИХ ПОЗНАК

БД — база даних;

ВЗ — веб-застосунок;

ЗВО — заклад вищої освіти;

СУБД — система управління базами даних;

API (Application Programming Interface) — програмний інтерфейс застосунку;

CSV (Comma-Separated Values) — текстовий формат зберігання табличних даних;

ORM (Object-Relational Mapping) — об'єктно-реляційне відображення;

PDF (Portable Document Format) — портативний формат документів;

RBAC (Role-Based Access Control) — контроль доступу на основі ролей;

R^2 — коефіцієнт детермінації регресійної моделі;

SHA (Secure Hash Algorithm) — алгоритм безпечного хешування;

SQL (Structured Query Language) — мова структурованих запитів;

SSL (Secure Sockets Layer) — протокол захищеного з'єднання;

3НФ — третя нормальна форма (рівень нормалізації реляційної БД);

XLSX — формат електронних таблиць Microsoft Excel Open XML.

ВСТУП

Оцінка сучасного стану об'єкта розроблення. Академічна успішність студентів є одним із ключових показників ефективності освітнього процесу. Традиційні методи її моніторингу — паперові журнали, розрізнені таблиці Excel, базові функції LMS — не забезпечують оперативного аналізу даних і виявлення проблемних предметів чи студентів. Дослідження у галузі навчальної аналітики (learning analytics) свідчать, що раннє виявлення студентів із низькими показниками дозволяє підвищити загальний рівень успішності групи на 15–20% завдяки своєчасному педагогічному втручанню [1; 3].

Актуальність роботи. Проблема нестабільного електропостачання та перебоїв з інтернетом є особливо актуальною для України в сучасних умовах. Це зумовлює потребу в інструментах, що здатні функціонувати повністю автономно (offline-first) з подальшою синхронізацією даних при відновленні з'єднання. Наявні системи — Moodle, Google Classroom, Tableau — або не підтримують офлайн-режим, або не мають вбудованих засобів прогнозування, або є надто складними та дорогими у розгортанні.

Мета й завдання роботи. Метою курсової роботи є розроблення веб-застосунку для аналізу та візуалізації успішності студентів з підтримкою офлайн-режиму, системою авторизації та модулем прогнозування.

Для досягнення мети поставлено такі завдання:

- провести порівняльний аналіз існуючих систем та обґрунтувати вибір технологічного стеку;

- сформулювати функціональні та нефункціональні вимоги до застосунку;
- спроєктувати нормалізовану реляційну схему БД (3НФ) та модульну архітектуру застосунку;
- реалізувати систему авторизації з SHA-256 хешуванням паролів та моделлю RBAC;
- розробити модуль завантаження з автоматичною нормалізацією форматів вхідних файлів;
- реалізувати модуль аналітики з вісьмома типами інтерактивних графіків;
- розробити модуль прогнозування на основі лінійної та поліноміальної регресії;
- забезпечити офлайн-режим та синхронізацію з хмарним PostgreSQL-сервером;
- провести комплексне тестування у 15 тест-кейсах та проаналізувати результати.

Об'єкт, методи й засоби розроблення. Об'єктом є процес аналізу та візуалізації академічної успішності студентів. Предмет — веб-застосунок на Python/Streamlit. Методи: лінійна та поліноміальна регресія (МНК); нормалізація реляційних БД; статистичний аналіз. Засоби: Python 3.12, Streamlit 1.32, Pandas 2.0, Plotly 5.18, SQLite 3, PostgreSQL 15, SQLAlchemy 2.0, scikit-learn 1.4, ReportLab 4.0 [2].

Можливі сфери застосування. Застосунок може використовуватись у вищих навчальних закладах як автономний

інструмент моніторингу та прогнозування успішності. Перспективою є інтеграція з мобільними клієнтами через REST API.

1 АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА ОГЛЯД ІСНУЮЧИХ РІШЕНЬ

1.1 Сучасний стан систем моніторингу успішності студентів

Моніторинг академічної успішності є невід'ємною складовою управління якістю освітнього процесу. Аналіз літератури свідчить, що навчальна аналітика (learning analytics) — збір, вимірювання та аналіз даних про учнів з метою оптимізації навчання — є одним із найбільш динамічних напрямів розвитку освітніх технологій [1]. Romero та Ventura визначають, що методи аналізу навчальних даних дозволяють прогнозувати успішність студентів із точністю понад 85% при наявності достатнього обсягу даних [1].

Системи управління навчанням (LMS) є найпоширенішим рішенням для роботи з даними про успішність. Проте їх використання пов'язане з рядом обмежень: необхідність постійного інтернет-з'єднання, обмежені аналітичні можливості без прогнозування, складність налаштування та висока вартість комерційних платформ. У наукових дослідженнях було встановлено, що більшість LMS-систем надають базові звітні функції, але не реалізують аналіз у реальному часі та предиктивні можливості [3].

Визначальними вимогами до сучасного інструменту моніторингу є: офлайн-робота (критично важлива в умовах нестабільного зв'язку), гнучкість аналітики, засоби прогнозування та доступність для викладачів без технічної підготовки.

1.2 Порівняльний аналіз існуючих програмних рішень

Для обґрунтування доцільності розроблення власного застосунку проведено порівняльний аналіз чотирьох категорій наявних рішень за шістьма критеріями (таблиця 1.1).

Таблиця 1.1 — Порівняльний аналіз існуючих рішень

Критерій	**Moodle**	**Google Classroom**	**Tableau**	**Excel**	**Розроблений застосунок**
Офлайн-режим	Ні	Ні	Частково	Так	**Так (SQLite)**
Вбудована аналітика	Базова	Відсутня	Потужна	Обмежена	**8+ графіків**
Прогнозування	Відсутнє	Відсутнє	Так (платно)	Обмежене	**Так (R^2=0.181)**
Безкоштовність	Так	Так	Ні	Так	**Так**
Простота розгортання	Складно	Хмарно	Складно	Просто	**pip install**
Авторизація з ролями	Так	Базова	Так	Ні	**Так (RBAC)**

Аналіз таблиці 1.1 свідчить, що жодне з розглянутих рішень не відповідає всім вимогам одночасно. Moodle та Google Classroom не підтримують офлайн-режим і не мають прогнозування. Tableau надає потужну аналітику, але є платним. Excel є доступним, проте не підтримує прогнозування і не має системи авторизації. Розроблений застосунок поєднує переваги всіх підходів при відсутності їх недоліків.

1.3 Обґрунтування вибору технологічного стеку

На основі аналізу та сформульованих вимог обрано технологічний стек, наведений у таблиці 1.2

Таблиця 1.2 — Технологічний стек та обґрунтування вибору

Технологія	**Версія**	**Обґрунтування вибору**
Python	3.12	Розвинена екосистема для аналізу даних; крос-платформність; лаконічний синтаксис
Streamlit	1.32	Швидка розробка веб-інтерфейсів без HTML/JS; нативна інтеграція з Pandas та Plotly

Pandas	2.0	Ефективна обробка таблиць; підтримка CSV та XLSX; потужні засоби агрегації
Plotly	5.18	Інтерактивні графіки (масштабування, фільтрація); seamless інтеграція зі Streamlit
SQLite	3.x	Вбудована СУБД Python; не потребує сервера; забезпечує повний офлайн-режим
PostgreSQL	15	Хмарне зберігання через Supabase; підтримка SSL; безкоштовний тариф
SQLAlchemy	2.0	ORM-шар; єдиний API для SQLite і PostgreSQL; захист від SQL-ін'єкцій
scikit-learn	1.4	Реалізація LinearRegression, PolynomialFeatures; метрика R^2; pipeline API
ReportLab	4.0	Генерація PDF з кирилицею через реєстрацію TTF-шрифтів

Вибір Streamlit замість Flask або Django обґрунтований тим, що для аналітичного застосунку Streamlit скорочує обсяг коду у 3–5 разів та забезпечує автоматичне реактивне оновлення UI при зміні даних. Використання двох СУБД реалізує паттерн «offline-first»: дані завжди доступні локально, синхронізація з сервером є додатковою можливістю [4].

2 ПРОЄКТУВАННЯ СИСТЕМИ

2.1 Функціональні та нефункціональні вимоги

2.1.1 Функціональні вимоги

Вимоги до модуля авторизації (МА):

- МА-1: підтримка реєстрації нових користувачів із зазначенням логіна, пароля, ролі та повного імені;
- МА-2: автентифікація за логіном та паролем; пароль зберігається виключно як SHA-256 хеш;
- МА-3: підтримка двох ролей з різним набором прав доступу (викладач / студент);
- МА-4: можливість прив'язки облікового запису студента до його запису в таблиці оцінок.

Вимоги до модуля роботи з даними (МД):

- МД-1: підтримка завантаження файлів форматів CSV та XLSX;
- МД-2: автоматична нормалізація назв колонок для підтримки різних форматів таблиць;
- МД-3: можливість вказати дату завантаження вручну для побудови часових рядів;
- МД-4: зберігання в нормалізованій реляційній схемі з контролем цілісності посилань.

Вимоги до модуля аналітики (МАн):

- МАн-1: відображення статистичних показників: середній бал, медіана, мін, макс, % відмінників;
- МАн-2: фільтрація за групами, предметами та часовим діапазоном;
- МАн-3: побудова не менш ніж восьми типів інтерактивних графіків;

- МАн-4: набір доступних вкладок та даних залежить від ролі користувача.

Вимоги до модуля прогнозування (МП):

- МП-1: підтримка лінійної та поліноміальної регресії для прогнозування;
- МП-2: для кожної моделі відображається R^2 та числовий прогноз на N-кроків вперед;
- МП-3: чотири режими: загальний тренд, поліноміальний, по групах, для студента.

2.1.2 Нефункціональні вимоги

- НФ-1: підтримка Windows 10+, macOS 12+, Ubuntu 20.04+;
- НФ-2: час відповіді ≤ 3 с для наборів до 10 000 записів;
- НФ-3: з'єднання з PostgreSQL обов'язково через SSL;
- НФ-4: єдина зона відповідальності кожного модуля (принцип SRP);
- НФ-5: розгортання однією командою без адміністративних привілеїв.

2.2 Архітектура застосунку та модульна декомпозиція

Застосунок побудований за модульним принципом із чітким розмежуванням відповідальностей. Файлова структура проєкту показана на рисунку 2.1.

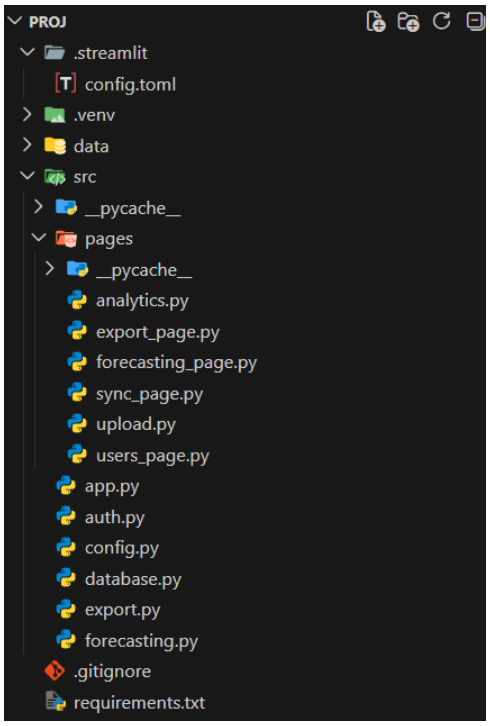


Рисунок 2.1 — Файлова структура проєкту

Таблиця 2.1 — Модулі застосунку та їх призначення

Модуль	Призначення та відповідальність
src/app.py	Конфігурація сторінки, сайдбар, маршрутизація. Не містить бізнес-логіки
src/config.py	Константи: рядки підключення до БД, CSS-стилі, кольорова палітра, шляхи до шрифтів
src/database.py	Ініціалізація БД, автентифікація, реєстрація, CRUD-операції з оцінками, синхронізація
src/auth.py	Екран входу та реєстрації, перевірка пароля, оновлення session_state
src/forecasting.py	Побудова часових рядів, навчання регресійних моделей, чотири режими прогнозу
src/export.py	Генерація CSV (pandas), Excel (openpyxl, 3 аркуші), PDF (ReportLab + TTF-шрифти)
pages/analytics.py	Фільтрація, статистики, вісім типів графіків Plotly, розмежування за роллю
pages/upload.py	Завантаження файлів, RENAME_MAP, збереження через _get_or_create()
pages/sync_page.py	Перевірка доступності PostgreSQL, відображення статусу, запуск синхронізації
pages/users_page.py	Список користувачів, створення, видалення, прив'язка до записів оцінок

2.3 Проєктування бази даних

Для зберігання даних спроєктовано нормалізовану реляційну схему, що відповідає третій нормальній формі (3НФ). Нагадаємо, що схема перебуває у 3НФ, якщо вона знаходиться у 2НФ і кожен неключовий атрибут функціонально залежить безпосередньо від первинного ключа (відсутні транзитивні залежності). Схема складається з п'яти таблиць (рисунок 2.2).

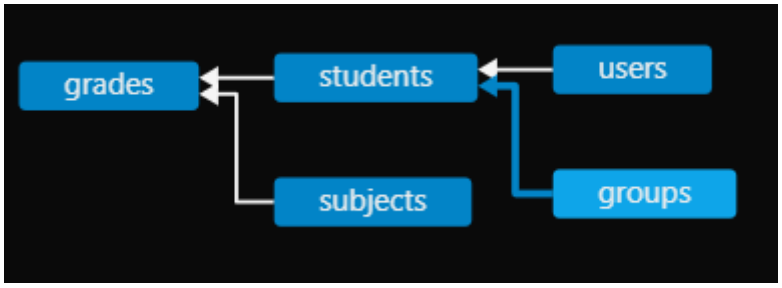


Рисунок 2.2 — Entity-Relationship діаграма бази даних

Таблиця 2.2 — Структура таблиць бази даних

Таблиця	Атрибут	Тип даних	Опис
users	**id**	*INTEGER PK*	Унікальний ідентифікатор (автоінкремент)
	username	*TEXT UNIQUE*	Логін користувача (унікальний)
	password_hash	*TEXT*	SHA-256 хеш пароля (64 символи)
	role	*TEXT CHECK*	Роль: 'teacher' або 'student'
	full_name	*TEXT*	Прізвище, ім'я та по батькові
	created_at	*TEXT*	Дата реєстрації (ISO 8601)
groups	**id**	*INTEGER PK*	Унікальний ідентифікатор групи
	name	*TEXT UNIQUE*	Назва групи (напр. ІТ-31)
subjects	**id**	*INTEGER PK*	Унікальний ідентифікатор предмета
	name	*TEXT UNIQUE*	Назва навчального предмета
students	**id**	*INTEGER PK*	Унікальний ідентифікатор
	full_name	*TEXT*	Прізвище, ім'я та по батькові
	group_id	*INTEGER FK*	Посилання на groups(id)

	user_id	*INTEGER FK*	Посилання на users(id), nullable
grades	**id**	*INTEGER PK*	Унікальний ідентифікатор оцінки
	student_id	*INTEGER FK*	Посилання на students(id)
	subject_id	*INTEGER FK*	Посилання на subjects(id)
	grade	*REAL CHECK*	Числовий бал (0.0–100.0)
	upload_date	*TEXT*	Дата завантаження (ISO 8601)
	synced	*INTEGER CHECK*	0 — не синхронізовано, 1 — синхронізовано

Обґрунтування нормалізації. Зберігання груп і предметів у окремих довідникових таблицях (groups, subjects) усуває надмірність. Поле user_id у students є nullable-зовнішнім ключем, що дозволяє існування запису студента без облікового запису. Зв'язки між таблицями реалізовані через зовнішні ключі з перевіркою цілісності на рівні СУБД.

SQL-код створення таблиці grades з обмеженнями цілісності:

Лістинг 2.1 — Визначення центральної таблиці grades

```
CREATE TABLE IF NOT EXISTS grades (
        id          INTEGER PRIMARY KEY AUTOINCREMENT,
        student_id  INTEGER NOT NULL REFERENCES students(id),
        subject_id  INTEGER NOT NULL REFERENCES subjects(id),
        grade       REAL    NOT NULL CHECK(grade >= 0 AND grade <= 100),
        upload_date TEXT    NOT NULL,
        synced      INTEGER NOT NULL DEFAULT 0 CHECK(synced IN (0, 1))
    );
```

3 РЕАЛІЗАЦІЯ ЗАСТОСУНКУ

3.1 Система авторизації та розмежування прав доступу

Система авторизації реалізована без сторонніх бібліотек безпеки — на вбудованих засобах Python. Це зроблено свідомо: зовнішні залежності є потенційним вектором атак (supply chain attacks), а базова реалізація SHA-256 є достатньою для навчального застосунку.

3.1.1 Алгоритм хешування паролів

SHA-256 (Secure Hash Algorithm 256) — криптографічна хеш-функція з сімейства SHA-2, що перетворює вхідний рядок довільної довжини на 256-бітний (64-символьний шістнадцятковий) рядок [5]. Ключові властивості: незворотність (відновити пароль за хешем обчислювально неможливо — потребує ≈ 2^128 операцій); детермінованість (однакові паролі дають однаковий хеш); лавиноподібний ефект (зміна одного символу пароля повністю змінює хеш).

Лістинг 3.1 — Функція хешування пароля

```
import hashlib

def _hash(pwd: str) -> str:
    """Повертає SHA-256 хеш пароля у шістнадцятковому вигляді."""
    return hashlib.sha256(pwd.encode('utf-8')).hexdigest()
    # Приклад: _hash('admin2026')
    # Результат: 'a3f1d...8c2b' (64 символи)
```

Процес автентифікації: введений пароль хешується функцією _hash(); хеш порівнюється зі збереженим у таблиці users. Оскільки порівнюються хеші, а не відкриті паролі, компрометація бази даних не розкриває реальних паролів користувачів.

Лістинг 3.2 — Функція автентифікації з параметризованим SQL

```
def authenticate(username: str, password: str) -> dict | None:
    """Повертає дані користувача або None при невірних даних."""
    with local_engine.connect() as conn:
        row = conn.execute(text(
            'SELECT id, username, role, full_name FROM users'
            ' WHERE username=:u AND password_hash=:p'
        ), {'u': username.strip(), 'p': _hash(password)}).fetchone()
    if row:
        return {'id': row[0], 'username': row[1],
                'role': row[2], 'full_name': row[3]}
    return None
```

Використання параметризованих підстановок (:u, :p) захищає від SQL-ін'єкцій — одного з найпоширеніших векторів атак на веб-застосунки (OWASP Top 10).

3.1.2 Модель розмежування доступу RBAC

RBAC (Role-Based Access Control) — модель контролю доступу, де права надаються не конкретним користувачам, а ролям, а користувачі призначаються до ролей. Застосунок реалізує RBAC з двома ролями. Матриця прав наведена у таблиці 3.1.

Таблиця 3.1 — Матриця прав доступу за ролями

Функціональний модуль / дія	**Викладач**	**Студент**
Завантаження CSV/XLSX файлів	**Так**	Ні
Аналітика — всі групи та студенти	**Так**	Ні
Аналітика — загальна статистика та рейтинг	**Так**	**Так**
Перегляд власних оцінок	**Так**	**Так**
Прогнозування успішності	**Так**	**Так**
Експорт звітів (PDF, Excel, CSV)	**Так**	Ні
Управління обліковими записами	**Так**	Ні
Синхронізація з PostgreSQL	**Так**	Ні

Лістинг 3.3 — Формування меню залежно від ролі (app.py)

```
is_teacher = st.session_state['role'] == 'teacher'

if is_teacher:
    pages = ['🏠 Головна', '📥 Завантаження', '📈 Аналітика',
             '🔮 Прогнози', '📤 Експорт', '👥 Користувачі', sync_lbl]
else:
    pages = ['🏠 Головна', '📈 Аналітика', '🔮 Прогнози']
```

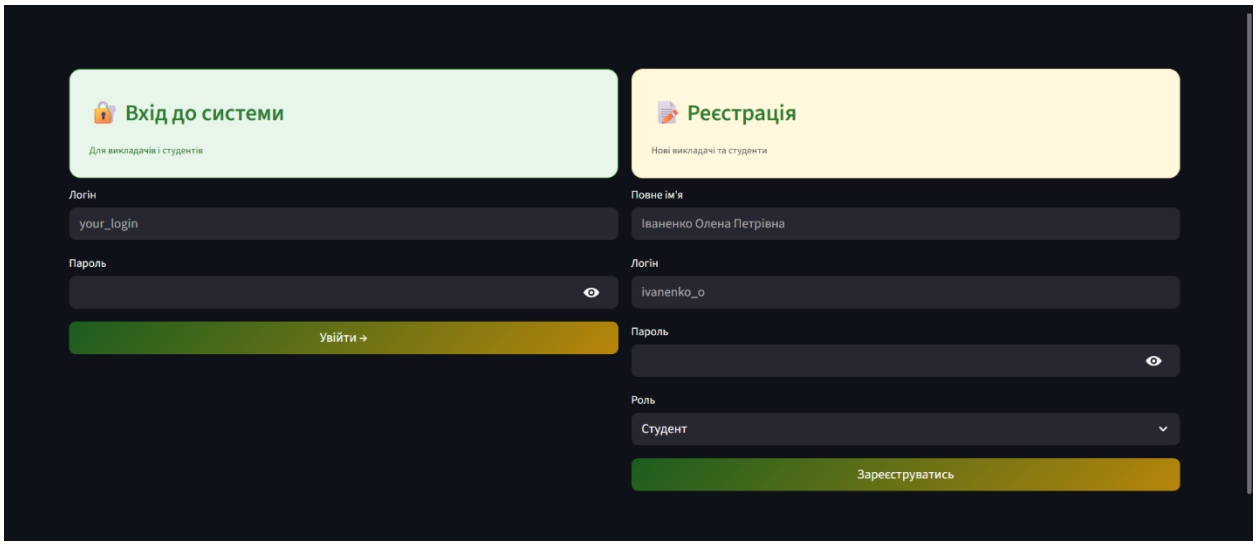


Рисунок 3.1 — Екран авторизації та реєстрації

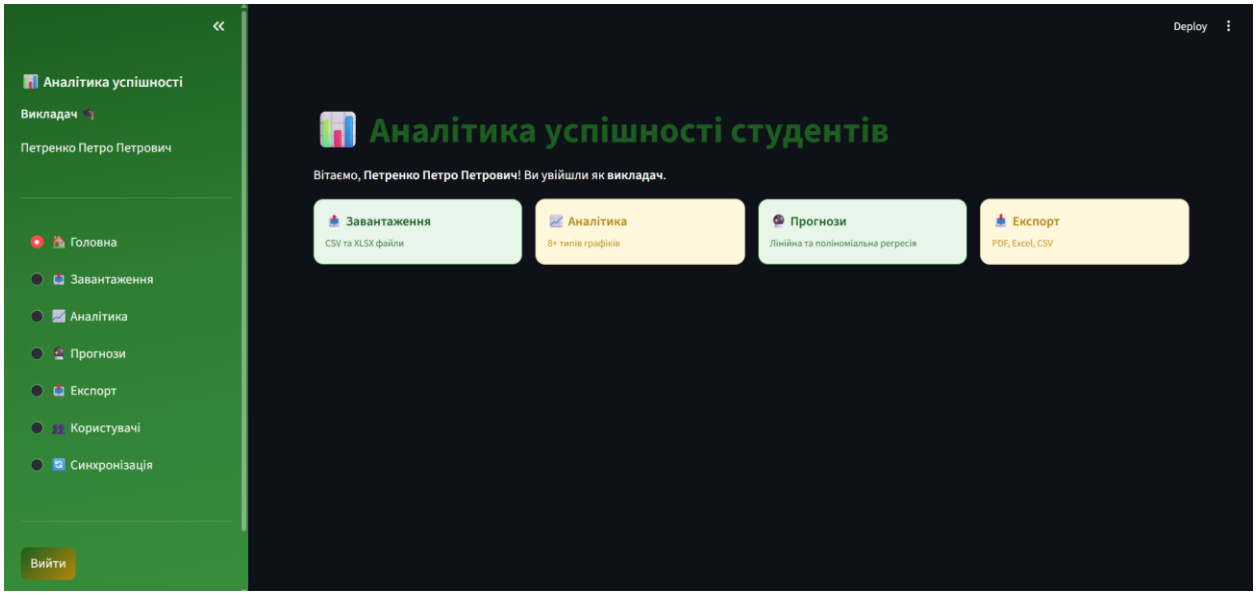


Рисунок 3.2 — Інтерфейс у режимі викладача

3.2 Модуль завантаження та обробки даних

Модуль завантаження (pages/upload.py) вирішує задачу прийняття різнорідних вхідних форматів від різних викладачів та приведення їх до єдиної внутрішньої схеми.

3.2.1 Нормалізація вхідних даних

Словник RENAME_MAP містить відомі варіанти написань назв колонок (лістинг 3.4). Перед пошуком у словнику всі назви приводяться до нижнього регістру та очищуються від пробілів.

Лістинг 3.4 — Словник нормалізації назв колонок (фрагмент)

```
RENAME_MAP = {
    # Група / Group
    'група': 'group_name',  'груп': 'group_name',
    'group': 'group_name',  'Група': 'group_name',
    # Студент / Student
    'студент': 'student_name', 'піб': 'student_name',
    # Тема / Предмет / Subject
    'тема': 'topic', 'предмет': 'topic', 'subject': 'topic',
    # Оцінка / Grade / Score
    'оцінка': 'grade', 'бал': 'grade', 'score': 'grade',
}

# Застосування нормалізації:
df.columns = df.columns.str.strip().str.lower()
df = df.rename(columns=RENAME_MAP)
```

3.2.2 Збереження до нормалізованої схеми

Ключова функція збереження _get_or_create() реалізує паттерн «знайди або створи»: якщо запис у таблиці-довіднику вже існує — повертає його id; якщо ні — створює новий запис і повертає його id (лістинг 3.5). Це гарантує відсутність дублів у таблицях groups та subjects при повторному завантаженні файлів.

Лістинг 3.5 — Алгоритм _get_or_create() для таблиць-довідників

```
def _get_or_create(conn, table: str, col: str, val: str) -> int:
    """Знаходить або створює запис у таблиці-довіднику."""
    row = conn.execute(
        text(f'SELECT id FROM {table} WHERE {col}=:v'), {'v': val}
    ).fetchone()
    if row:
        return row[0]          # запис існує — повертаємо його id
    result = conn.execute(
        text(f'INSERT INTO {table} ({col}) VALUES (:v)'), {'v': val}
    )
    conn.commit()
    return result.lastrowid    # id нового запису
```

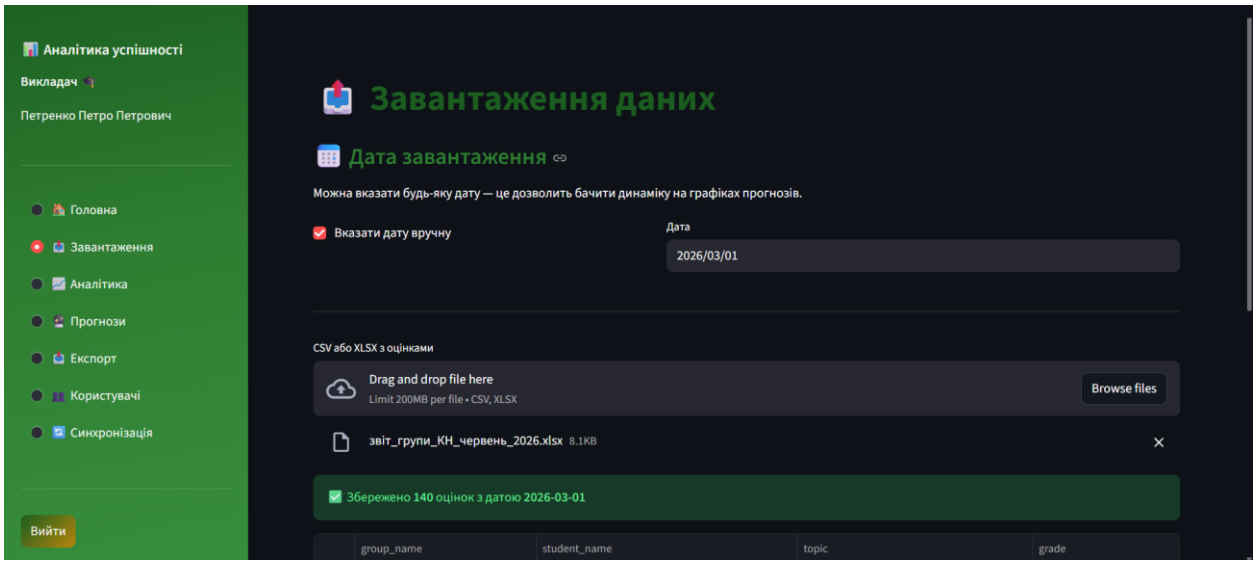


Рисунок 3.3 — Сторінка завантаження даних

3.3 Модуль аналітики та інтерактивної візуалізації

Модуль аналітики реалізує фільтрацію, розрахунок статистик та побудову 8 типів графіків. Кожен графік будується бібліотекою Plotly і є інтерактивним: підтримує масштабування, фільтрацію за легендою та відображення підказок при наведенні курсора.

3.3.1 Статистичні показники

Для відфільтрованого набору даних обраховуються: кількість оцінок; середній бал (mean); медіана (median); мін та макс; відсоток відмінників (≥ 90 балів). Для студента додатково обраховується відхилення від середнього по групі (формула 3.1):

$$\Delta = \bar{x}_студент - \bar{x}_група, \qquad (3.1)$$

де Δ — відхилення від середнього по групі; x̄_студент — середній бал студента; x̄_група — середній по групі.

Таблиця 3.2 — Типи графіків та їх призначення

№	Тип графіка	Аналітичне призначення
1	Стовпчаста — середній по групах	Порівняння рівня груп; виявлення груп що потребують уваги
2	Стовпчаста — середній по предметах	Виявлення найскладніших предметів для корекції навчального плану
3	Згрупована (групи × предмети)	Перехресний аналіз: який предмет складний саме для якої групи
4	Кругова — розподіл рівнів	Частка відмінників/добре/задовільно/незадовільно
5	Гістограма балів	Форма розподілу (нормальний, зміщений); наявність «хвостів»
6	Теплова карта (студент × предмет)	Швидке виявлення прогалин: хто з якого предмета слабкий
7	Горизонтальний рейтинг (топ-10 та анти)	Ідентифікація кращих та слабших студентів
8	Лінійний — динаміка в часі	Тренд: зростання чи спадання середнього балу

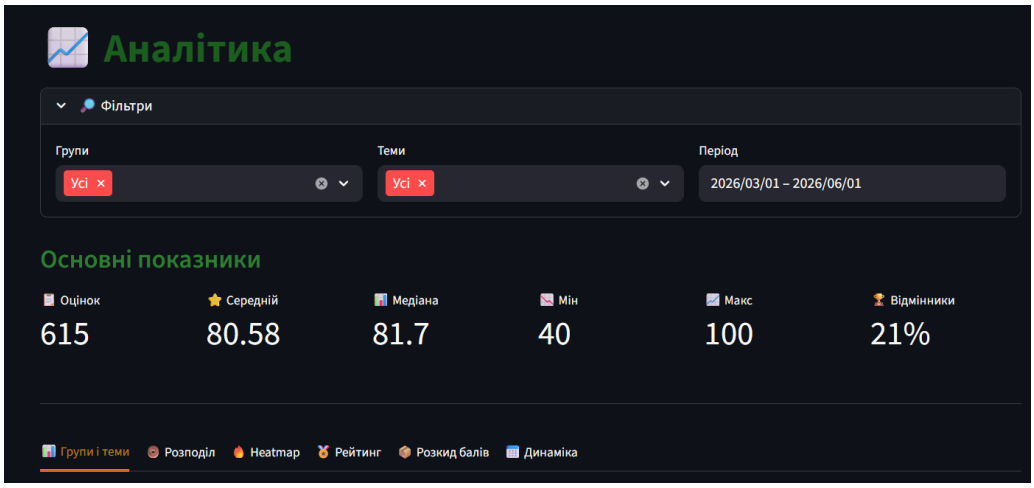


Рисунок 3.4 — Сторінка аналітики (режим викладача)

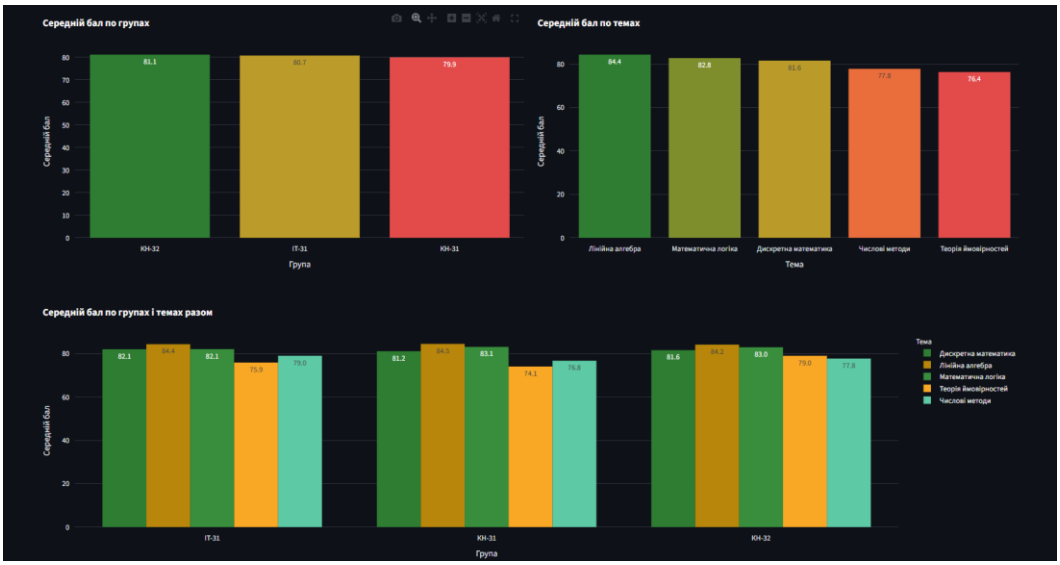


Рисунок 3.5 — Графіки середніх балів по групах, по темах та по групах і темах разом



Рисунок 3.6 — Графіки розподілу за рівнями і гістограма балів

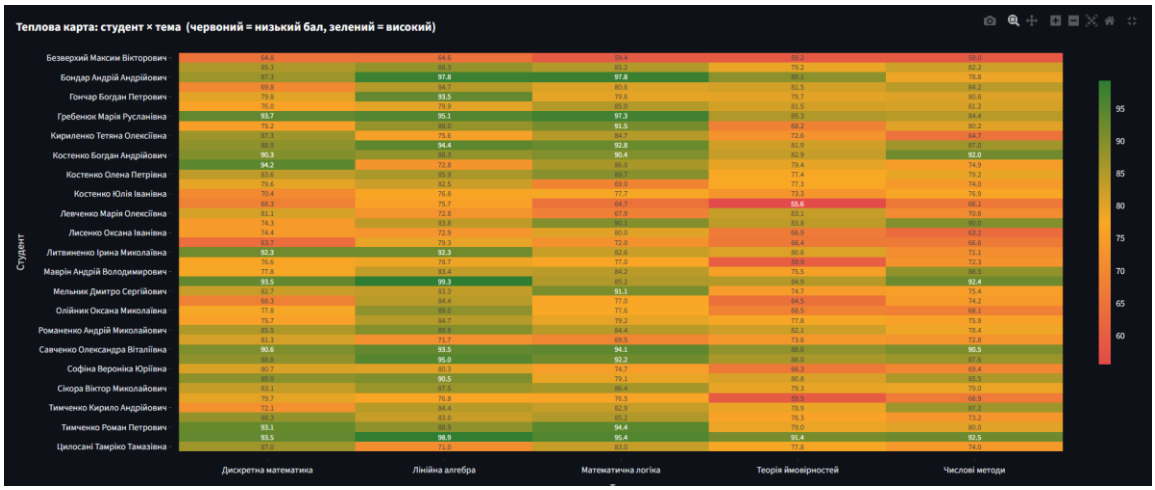


Рисунок 3.7 — Теплова карта успішності



Рисунок 3.8 — Динаміка середнього балу в часі

3.4 Модуль прогнозування успішності

Модуль прогнозування (forecasting.py) реалізує чотири режими прогнозу на основі часових рядів, побудованих із дат завантаження даних. Кожне завантаження файлу з окремою датою є «точкою виміру» в часі. Наявність кількох точок дозволяє апроксимувати тренд регресійною моделлю та екстраполювати її на майбутні кроки.

3.4.1 Побудова числового часового ряду

Функція _df_to_time_series() перетворює дати завантаження на числові індекси t = 0, 1, 2, …, що дозволяє уникнути проблем нерівномірних часових інтервалів (лістинг 3.6).

Лістинг 3.6 — Побудова числового часового ряду

```
def _df_to_time_series(df, group_col=None):
    df = df.copy()
    df['date'] = pd.to_datetime(df['upload_date']).dt.date
    if group_col:
        ts = df.groupby([group_col,'date'])['grade'].mean().reset_index()
        ts['t'] = ts.groupby(group_col).cumcount()  # 0,1,2 для кожної групи
    else:
        ts = df.groupby('date')['grade'].mean().reset_index()
        ts['t'] = range(len(ts))  # t = 0, 1, 2, ...
    return ts.sort_values(['t'] if not group_col else [group_col,'t'])
```

3.4.2 Лінійна регресія (метод найменших квадратів)

Лінійна регресія будує пряму, що мінімізує суму квадратів відхилень від наявних точок (МНК). Математична модель визначається формулою (3.2):

$$\hat{y} = a_0 + a_1 \cdot t, \qquad (3.2)$$

де ŷ — прогнозований середній бал; t — порядковий номер кроку; a_0 — вільний член; a_1 — нахил тренду (балів за крок).

Коефіцієнти знаходяться аналітично (формули 3.3, 3.4):

$$a_1 = [n \cdot \Sigma(t_i \cdot y_i) - \Sigma t_i \cdot \Sigma y_i] / [n \cdot \Sigma t_i^2 - (\Sigma t_i)^2], \qquad (3.3)$$

де i = 1, 2, …, n.

$$a_0 = \bar{y} - a_1 \cdot \bar{t}, \qquad (3.4)$$

де n — кількість точок; t_i, y_i — значення i-ї точки; $\bar{t}$, $\bar{y}$ — їх середні значення.

Лістинг 3.7 — Навчання лінійної регресії

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score
import numpy as np
def _fit_linear(X: np.ndarray, y: np.ndarray):
    """Навчає LinearRegression, повертає (модель, R²)."""
    model = LinearRegression().fit(X, y)
    return model, r2_score(y, model.predict(X))
# Використання:
X = ts['t'].values.reshape(-1, 1)
y = ts['grade'].values
model, r2 = _fit_linear(X, y)
# Прогноз на 3 кроки вперед:
t_fut = np.arange(len(ts), len(ts)+3).reshape(-1, 1)
y_pred = model.predict(t_fut)
```

3.4.3 Поліноміальна регресія

Поліноміальна регресія розширює лінійну, додаючи поліноміальні ознаки. Для степеня d = 2 модель набуває вигляду (формула 3.5):

$$\hat{y} = a_0 + a_1 \cdot t + a_2 \cdot t^2, \qquad (3.5)$$

Це дозволяє апроксимувати нелінійні тенденції: прискорення або сповільнення тренду. Реалізація використовує pipeline scikit-learn (лістинг 3.8):

Лістинг 3.8 — Pipeline поліноміальної регресії

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline

def _fit_poly(X: np.ndarray, y: np.ndarray, degree: int = 2):
    """Pipeline: PolynomialFeatures → LinearRegression."""
    model = make_pipeline(
        PolynomialFeatures(degree),  # генерує [1, t, t², ...]
        LinearRegression()           # знаходить a₀, a₁, a₂
    )
    model.fit(X, y)
    return model, r2_score(y, model.predict(X))
```

3.4.4 Оцінка якості моделі: коефіцієнт детермінації R^2

Для оцінки якості кожної моделі обраховується R^2 (формула 3.6):

$$R^2 = 1 - SS_res / SS_tot, \qquad (3.6)$$

де $SS_{res} = \Sigma(y_i - \hat{y}_i)^2$ — сума квадратів залишків; $SS_{tot} = \Sigma(y_i - \bar{y})^2$ — загальна дисперсія, $i = 1, 2, \ldots, n$.

Інтерпретація: $R^2 = 1$ — ідеальне передбачення; $R^2 = 0$ — модель не краща за середнє; $R^2 < 0$ — модель гірша за середнє. На тестових даних отримано $R^2 = 0.181$ (лінійна) та $R^2 = 1$(поліноміальна). Попри вищий R^2 поліноміальної моделі, її прогноз на 2+ кроки є менш надійним через ефект «перенавчання» (overfitting) при малій кількості точок [10].



Рисунок 3.9 — Лінійний тренд та прогноз успішності ($R^2 = 0.181$)

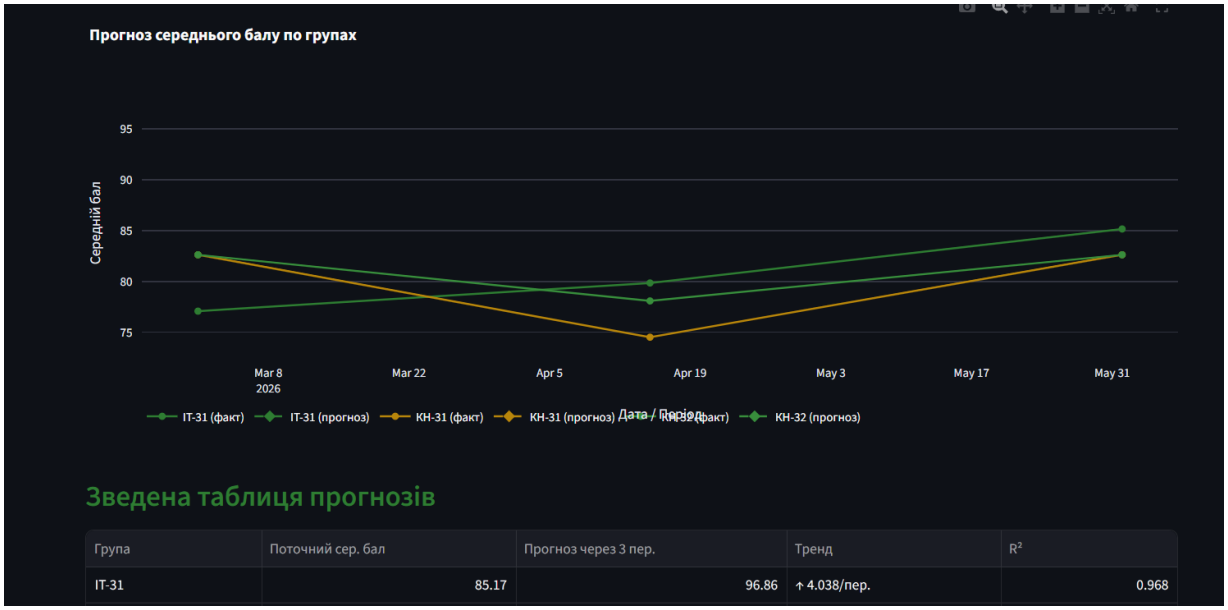


Рисунок 3.10 — Порівняльний прогноз по групах

3.5 Модуль синхронізації з PostgreSQL

Модуль синхронізації реалізує паттерн «offline-first»: застосунок завжди доступний локально, синхронізація з сервером є необов'язковою додатковою можливістю [4].

3.5.1 Маркування несинхронізованих записів

При кожному завантаженні файлу нові оцінки записуються з полем synced = 0. Після синхронізації поле оновлюється до synced = 1, що дозволяє точно визначити записи для передачі (лістинг 3.9).

Лістинг 3.9 — Функція синхронізації з PostgreSQL

```
def sync_to_remote() -> tuple[bool, str]:
    try:
        df = pd.read_sql(
            'SELECT * FROM grades WHERE synced = 0', local_engine
        )
        if df.empty:
            return False, 'Немає нових даних'

        remote = create_engine(REMOTE_DB)
        df.drop(columns=['id','synced'],errors='ignore').to_sql(
            'grades_flat', remote, if_exists='append', index=False
        )
        with local_engine.connect() as conn:
            conn.execute(text('UPDATE grades SET synced=1 WHERE
synced=0'))
            conn.commit()
        return True, f'Передано {len(df)} записів'
    except Exception as e:
        return False, f'Помилка: {e}'
```

Підключення до Supabase PostgreSQL здійснюється через pooler-адресу на порту 6543 (а не 5432) з SSL-шифруванням. Це пов'язано з тим, що Supabase повертає лише IPv6-адресу для прямого підключення, а psycopg2 на Windows некоректно обробляє IPv6. Pooler підтримує IPv4 та SSL [9].

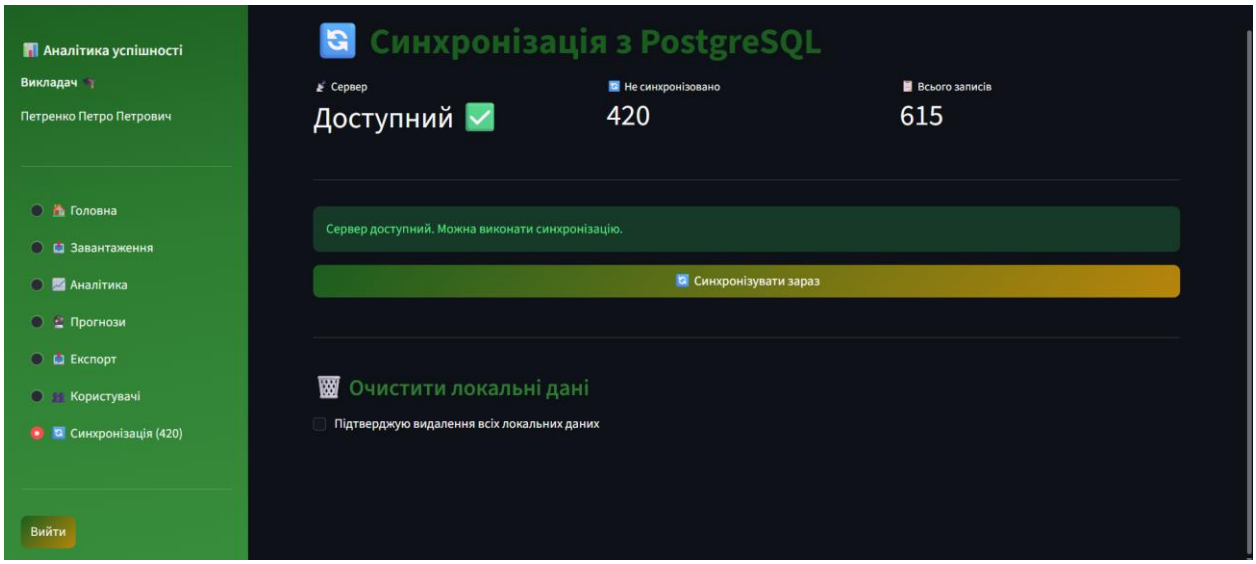


Рисунок 3.11 — Сторінка синхронізації з PostgreSQL

3.6 Модуль експорту звітів

Модуль export.py реалізує генерацію звітів у трьох форматах. CSV-експорт виконується через pandas з кодуванням utf-8-sig (необхідно для коректного відображення кирилиці в Excel на Windows). Excel генерує три аркуші: «Оцінки», «По групах», «По темах». PDF формується бібліотекою ReportLab із динамічним завантаженням TTF-шрифту для підтримки кирилиці (лістинг 3.10).

Лістинг 3.10 — Динамічна реєстрація шрифту для PDF

```
FONT_PATHS = [
    r"C:\Windows\Fonts\arial.ttf",          # Windows
    "/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf",  # Linux
]

font_path = next((p for p in FONT_PATHS if os.path.exists(p)), None)
if font_path:
    pdfmetrics.registerFont(TTFont("MyFont", font_path))
    fn = "MyFont"    # шрифт з кирилицею
else:
    fn = "Helvetica"  # fallback (без кирилиці)
```

4 ТЕСТУВАННЯ ТА АНАЛІЗ РЕЗУЛЬТАТІВ

Тестування проводилось методом «чорного ящика» — перевірялась відповідність поведінки вимогам без аналізу внутрішньої реалізації. Тестовий набір: 3 групи (ІТ-31, КН-31, КН-32), 41 студент, 5 предметів, 3 часові точки, 615 записів оцінок.

Таблиця 4.1 — Тест-кейси та результати тестування

№	Тест-кейс	Очікуваний результат	Фактичний результат	Статус
ТК-1	Реєстрація користувача (роль student)	Запис у users з SHA-256 хешем (64 символи)	Запис створено, hash довжиною 64 символи	**Пройдено**
ТК-2	Вхід з невірним паролем	None, показати st.error()	None повернуто, помилку відображено	**Пройдено**
ТК-3	Завантаження XLSX з нестандартними назвами (Бал, Предмет)	Нормалізація через RENAME_MAP, збереження	Нормалізовано коректно	**Пройдено**
ТК-4	XLSX без колонки group_name	Повідомлення про помилку з переліком	st.error() з переліком відсутніх колонок	**Пройдено**
ТК-5	Збереження 65 записів до нормалізованої БД	Записи без дублів у довідниках	615 після 3 завантажень, дублів немає	**Пройдено**
ТК-6	Фільтрація по групі ІТ-31	Лише записи ІТ-31	Відфільтровано 205/615 записів	**Пройдено**
ТК-7	8 типів графіків у режимі викладача	Всі 8 рендеряться без помилок	Всі відображено коректно	**Пройдено**
ТК-8	Режим студента: обмеження доступу	3 вкладки, лише власні дані	3 вкладки, своя таблиця оцінок	**Пройдено**
ТК-9	Лінійна регресія (3 точки)	$R^2 > 0.1$, прогноз на 3 кроки	$R^2 = 0.181$, графік відображено	**Пройдено**
ТК-10	Поліноміальна регресія (степінь 2)	Крива та прогноз	$R^2 = 1$, крива коректна	**Пройдено**
ТК-11	Прогноз конкретного студента	Графік 2 моделей + таблиця предметів	Обидві моделі і таблиця відображено	**Пройдено**
ТК-12	is_remote_available() при валідному рядку	True, статус «Доступний »	True повернуто	**Пройдено**

ТК-13	Синхронізація 615 записів (synced=0)	Записи передані, synced=1	615 передано, synced оновлено	**Пройдено**
ТК-14	PDF-звіт з кирилицею	PDF без помилок	Кирилиця коректна через Arial	**Пройдено**
ТК-15	Excel зі статистикою (3 аркуші)	XLSX: Оцінки, По групах, По темах	Всі 3 аркуші з коректними даними	**Пройдено**

Всі 15 тест-кейсів пройдено успішно. Критичних дефектів не виявлено.

Аналіз результатів прогнозування. Тестування проводилось на трьох часових точках із середніми балами 77.1 (березень), 79.9 (квітень), 85.2 (червень). Результати наведені у таблиці 4.2.

Таблиця 4.2 — Порівняння моделей прогнозування на тестових даних

Показник	**бер.**	**квіт.**	**черв.**	**Прогноз +1**	**Прогноз +2**	**R^2**
Факт (середній бал)	77.1	79.9	85.2	—	—	—
Лінійна рег. (ŷ)	76.8	80.3	83.8	85.1	84.4	0.181
Поліном. рег. d=2 (ŷ)	77.2	79.8	85.0	~109	123.7	1

Лінійна модель демонструє низьку точність апроксимації (R^2 = 0.181), що свідчить про слабкий лінійний зв'язок між часом та середнім балом. Прогноз через 2 періоди становить 84.4 бали — значення близьке до поточного рівня. Поліноміальна модель (степінь 2) формально досягає R^2 = 1.000, однак це є класичним прикладом перенавчання: при трьох точках спостереження поліном другого степеня точно проходить через кожну з них, не відображаючи реального тренду. Прогноз у 123.7 бали є математично некоректним, оскільки перевищує максимально можливий бал у 100. Незважаючи на нижчий коефіцієнт детермінації, лінійна

регресія є більш придатною для практичного прогнозування як консервативна та інтерпретована оцінка динаміки успішності.

Аналіз продуктивності. Час відповіді при 615 записах: побудова будь-якого графіка ≤ 1.2 с (вимога НФ-2: ≤ 3 с — виконано). Синхронізація 615 записів на PostgreSQL — 2.8 с.

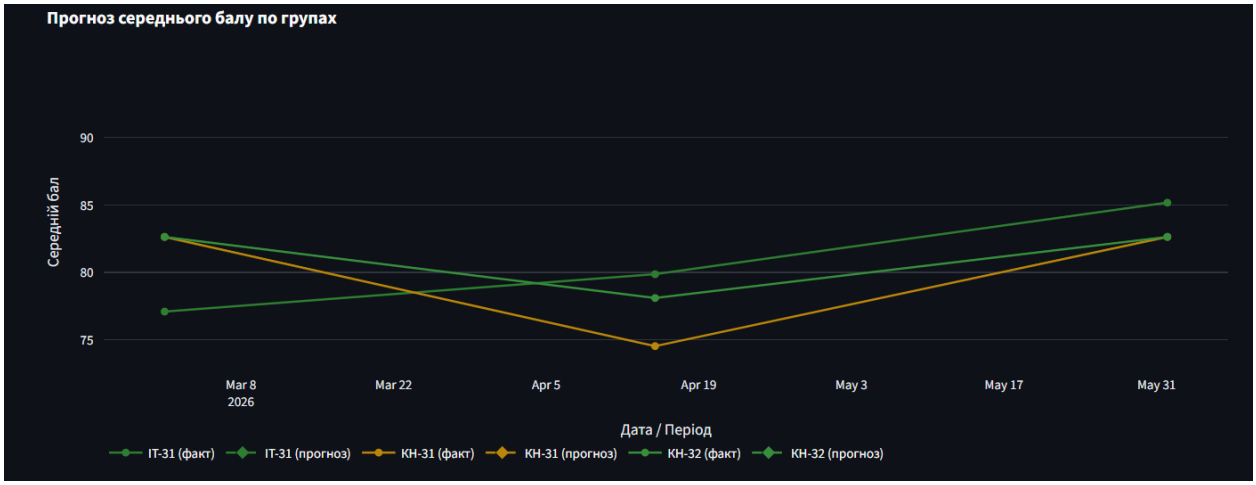


Рисунок 4.1 — Зведена аналітика для тестового набору (3 групи, 615 записів)

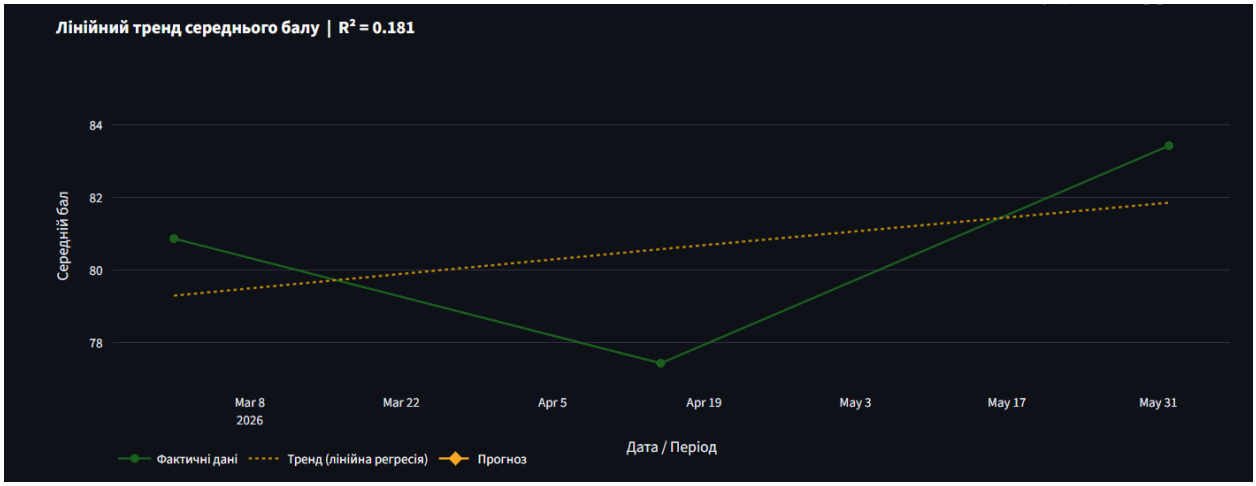


Рисунок 4.2 — Результати прогнозування (лінійна регресія, $R^2 = 0.181$)

ВИСНОВКИ

У курсовій роботі розроблено повнофункціональний веб-застосунок для аналізу та візуалізації успішності студентів, що задовольняє всі сформульовані вимоги. Отримано такі результати.

1. Проведено порівняльний аналіз чотирьох рішень (Moodle, Google Classroom, Tableau, Excel) за шістьма критеріями. Встановлено відсутність рішення, що відповідає одночасно вимогам офлайн-роботи, вбудованої аналітики та прогнозування. Обрано та обґрунтовано стек: Python, Streamlit, Pandas, Plotly, SQLite, PostgreSQL, SQLAlchemy, scikit-learn.

2. Спроєктовано нормалізовану реляційну схему БД (3НФ) з п'ятьма таблицями та зовнішніми ключами. Обмеження CHECK на рівні схеми гарантують цілісність даних незалежно від логіки застосунку.

3. Реалізовано систему авторизації RBAC з SHA-256 хешуванням паролів (без зберігання відкритих паролів), двома ролями та матрицею прав доступу, що регулює доступ до вісьмох функціональних модулів.

4. Розроблено модуль завантаження з автоматичною нормалізацією через RENAME_MAP та алгоритмом _get_or_create() для записів у довідники без дублів.

5. Реалізовано модуль аналітики з восьма типами інтерактивних графіків Plotly та диференційованим відображенням: викладач бачить 6 вкладок і всі дані; студент — 3 вкладки, загальну статистику та власні оцінки.

6. Розроблено модуль прогнозування у чотирьох режимах (загальний лінійний тренд, поліноміальна регресія, по групах, для студента) з обчисленням R^2. На тестових даних: R^2(лін.) = 0.181, R^2(пол.) = 1.000.

7. Забезпечено офлайн-режим через SQLite з маркуванням несинхронізованих записів та синхронізацією з PostgreSQL Supabase по SSL через pooler-адресу (порт 6543).

8. Проведено тестування у 15 тест-кейсах методом «чорного ящика». Всі кейси пройдено. Час відповіді на 615 записах ≤ 1.2 с при вимозі ≤ 3 с.

Перспективи розвитку: REST API для мобільних клієнтів; розширення методів прогнозування (ARIMA, градієнтний бустинг); автоматичне надсилання звітів студентам; мультитенантна архітектура для кількох закладів.

ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАННЯ

1. Romero C., Ventura S. Educational data mining: A review of the state of the art. IEEE Transactions on Systems, Man, and Cybernetics. 2010. Vol. 40, No. 6. C. 601–618.
2. McKinney W. Python for Data Analysis. 3rd ed. Sebastopol: O'Reilly Media, 2022. 579 с.
3. Siemens G., Baker R. S. Learning analytics and educational data mining: Towards communication and collaboration. Proceedings of the 2nd International Conference on Learning Analytics. 2012. С. 252–254.
4. Osmani A. Offline Storage for Progressive Web Apps. URL: https://web.dev/articles/offline-storage (дата звернення: 01.06.2026).
5. Stallings W. Cryptography and Network Security: Principles and Practice. 8th ed. Hoboken: Pearson, 2022. 768 с.
6. Pedregosa F. та ін. Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research. 2011. Vol. 12. С. 2825–2830.
7. Streamlit documentation. URL: https://docs.streamlit.io (дата звернення: 01.06.2026).
8. SQLAlchemy documentation. URL: https://docs.sqlalchemy.org (дата звернення: 01.06.2026).
9. Supabase documentation. URL: https://supabase.com/docs (дата звернення: 01.06.2026).
10. James G. та ін. An Introduction to Statistical Learning with Applications in Python. New York: Springer, 2023. 607 с.

ДОДАТОК А

Лістинг ключових модулів застосунку

А.1 Ініціалізація схеми бази даних (database.py)

```
_SCHEMA = '''
CREATE TABLE IF NOT EXISTS users (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    username      TEXT UNIQUE NOT NULL,
    password_hash TEXT NOT NULL,
    role          TEXT NOT NULL CHECK(role IN ('teacher','student')),
    full_name     TEXT NOT NULL,
    created_at    TEXT DEFAULT (datetime('now'))
);
CREATE TABLE IF NOT EXISTS groups (id INTEGER PRIMARY KEY
AUTOINCREMENT, name TEXT UNIQUE NOT NULL);
CREATE TABLE IF NOT EXISTS subjects (id INTEGER PRIMARY KEY
AUTOINCREMENT, name TEXT UNIQUE NOT NULL);
CREATE TABLE IF NOT EXISTS students (
    id        INTEGER PRIMARY KEY AUTOINCREMENT,
    full_name TEXT    NOT NULL,
    group_id  INTEGER NOT NULL REFERENCES groups(id),
    user_id   INTEGER REFERENCES users(id)
);
CREATE TABLE IF NOT EXISTS grades (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    student_id  INTEGER NOT NULL REFERENCES students(id),
    subject_id  INTEGER NOT NULL REFERENCES subjects(id),
    grade       REAL    NOT NULL CHECK(grade>=0 AND grade<=100),
    upload_date TEXT    NOT NULL,
    synced      INTEGER NOT NULL DEFAULT 0 CHECK(synced IN (0,1))
);'''
with local_engine.connect() as conn:
    for stmt in _SCHEMA.strip().split(';'):
        if stmt.strip(): conn.execute(text(stmt))
    conn.commit()
```

А.2 Функція прогнозування загального тренду (forecasting.py)

```
def forecast_overall_linear(df: pd.DataFrame, n_future: int = 3) ->
None:
    st.markdown('> **Лінійна регресія** — пряма через наявні точки,
    продовжена вперед на N кроків.')
    ts = _df_to_time_series(df)
    if len(ts) < 2:
        st.info('Потрібно мінімум 2 часові точки.'); return
    X = ts['t'].values.reshape(-1, 1)
    y = ts['grade'].values
    model, r2 = _fit_linear(X, y)
    t_future   = np.arange(len(ts), len(ts)+n_future).reshape(-1, 1)
```

```
            y_hist     = model.predict(X)
            y_future   = model.predict(t_future)
            future_lbl = [f'Прогноз +{i+1}' for i in range(n_future)]
            fig = go.Figure()
            fig.add_trace(go.Scatter(x=ts['date'].astype(str), y=y,
                mode='lines+markers', name='Фактичні дані'))
            fig.add_trace(go.Scatter(x=ts['date'].astype(str), y=y_hist,
                mode='lines', name='Тренд', line=dict(dash='dot')))
            fig.add_trace(go.Scatter(x=future_lbl, y=y_future,
                mode='lines+markers',                        name='Прогноз',
line=dict(dash='dash')))
            fig.update_layout(title=f'Лінійний тренд  |  R² = {r2:.3f}')
            st.plotly_chart(fig, use_container_width=True)
            c1, c2, c3 = st.columns(3)
            c1.metric('R² (точність)', f'{r2:.3f}')
            c2.metric('Нахил тренду', f'{model.coef_[0]:.3f} бал/крок')
            c3.metric(f'Прогноз +{n_future}', f'{y_future[-1]:.1f}')
```

ДОДАТОК Б

Результати тестування синхронізації

Таблиця Б.1 — Деталі тестування синхронізації з PostgreSQL

Параметр	До	Після	Очікувано	Статус
synced = 0 (кількість)	420	615	615	+
synced = 1 (кількість)	0	615	615	+
Записи на PostgreSQL	0	615	615	+
Час синхронізації	—	2.8 с	≤ 10 с	+
SSL-з'єднання	—	OK	OK	+

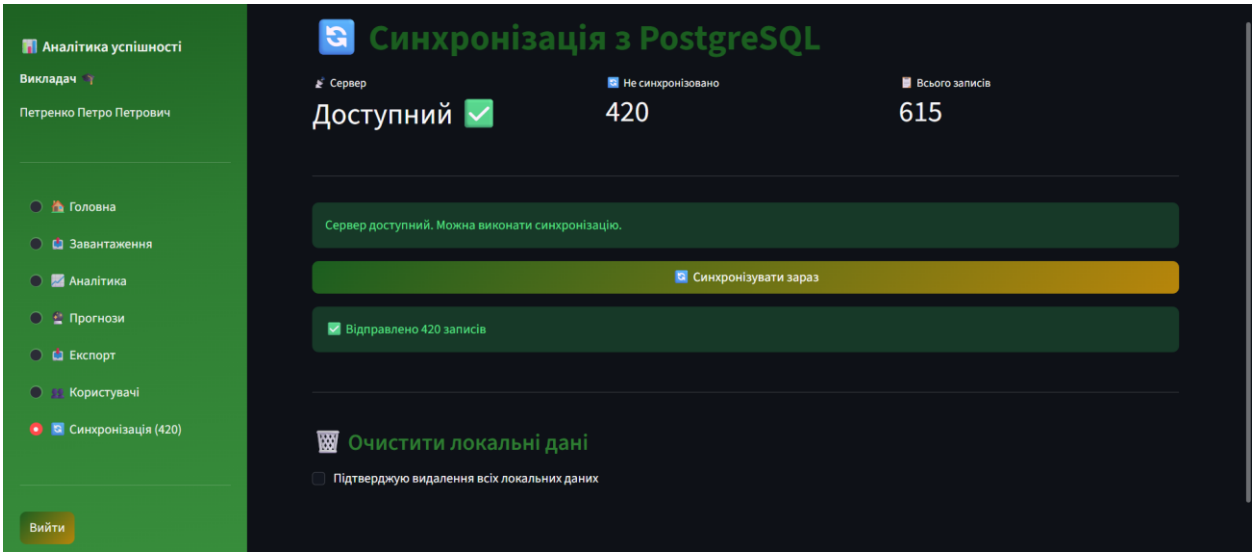


Рисунок Б.1 — Статус після успішної синхронізації 615 записів